Solution: Minimum Cost to Connect Sticks(medium)

Problem Statement

Given a collection of sticks with different lengths. To combine any two sticks, there's a cost involved, which is equal to the sum of their lengths.

Connect all the sticks into a single one with the minimum possible cost. Remember, once two sticks are combined, they form a single stick whose length is the sum of the lengths of the two original sticks.

Examples

1.
   - **Input:** [2, 4, 3]
   - **Expected Output:** 14
   - **Justification:** Combine sticks 2 and 3 for a cost of 5. Now, we have sticks [4,5]. Combine these at a cost of 9. Total cost = 5 + 9 = 14.
2.
   - **Input:** [1, 8, 2, 5]
   - **Expected Output:** 30
   - **Justification:** Combine sticks 1 and 2 for a cost of 3. Now, we have sticks [3, 8, 5]. Combine 3 and 5 for a cost of 8. Now, we have sticks [8,8]. Combine these for a cost of 16. Total cost = 3 + 8 + 16 = 27.
3.
   - **Input:** [5, 5, 5, 5]
   - **Expected Output:** 40
   - **Justification:** Combine two 5s for a cost of 10. Do this again for another cost of 10. Now, we have two sticks of 10 each. Combine these for a cost of 20. Total cost = 10 + 10 + 20 = 40.

Solution

The crux of the solution is to always combine the two shortest sticks in the collection. By doing this iteratively until only one stick remains, we can ensure the minimum possible cost. A priority queue (min-heap) is ideal for this task as it allows us to always efficiently retrieve and combine the two shortest sticks, and then insert the resulting combined stick back for further combinations.

Detailed Steps:

1. **Initialization**: Begin by inserting all the stick lengths into a min-heap. This structure ensures that the smallest stick lengths are always easily accessible at the top.
2. **Combination**: Until we have more than one stick left in the heap, proceed with the following steps:
   - Remove the two smallest stick lengths from the heap (these are the two at the top).
   - Combine these two sticks, which results in a new stick whose length is the sum of the two original ones. Add this combination cost to a running total.
   - Insert the new combined stick length back into the heap. This is done to consider this stick for future combinations.
3. **Final Stick**: After all combinations are complete, we'll be left with just one stick in the heap, which represents the combined length of all original sticks. The running total we kept during the combination steps represents the minimum cost to connect the sticks.
4. **Return**: Lastly, the accumulated cost is returned as the answer.

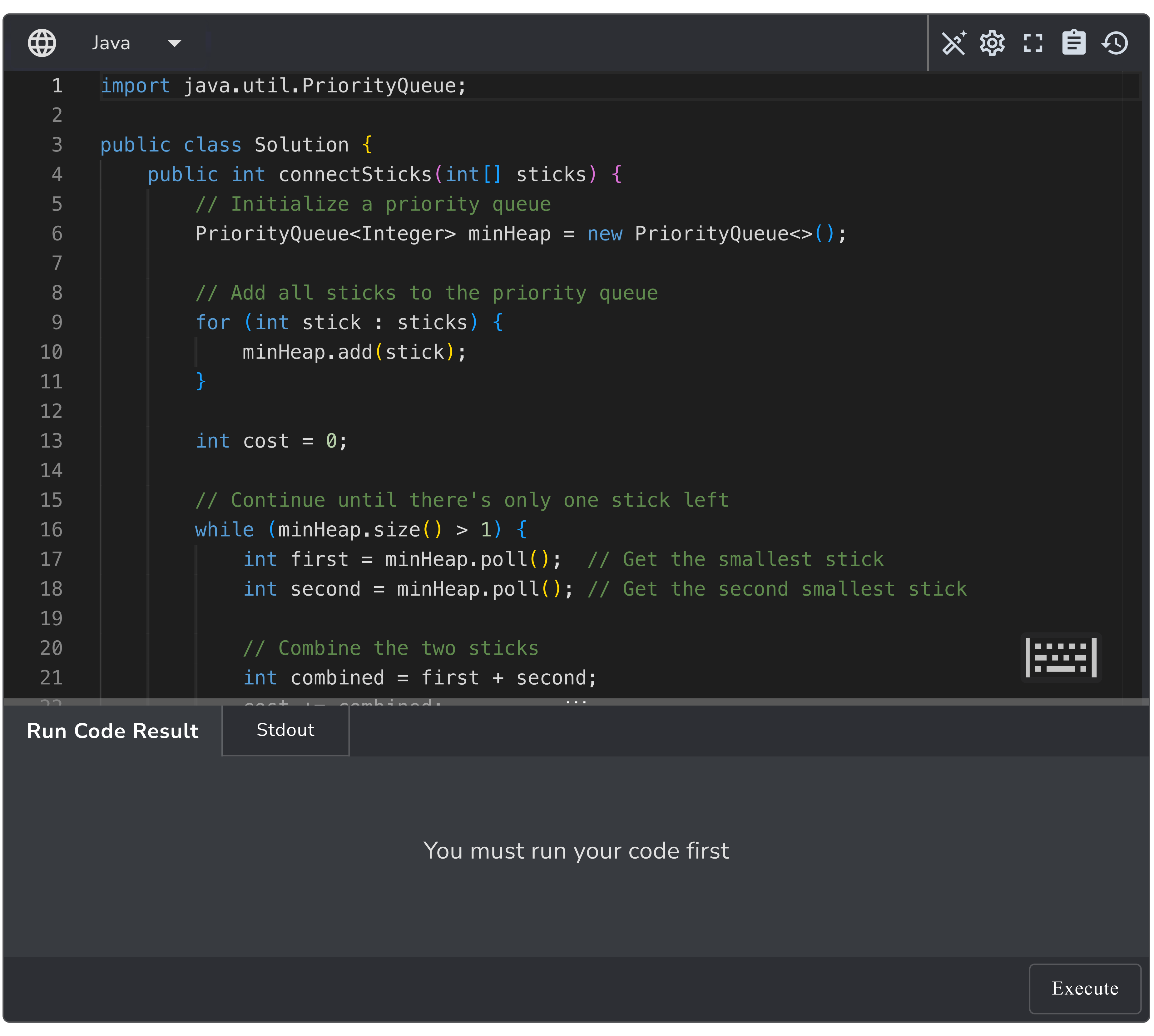
The reason this approach is optimal is that, at each step, we're combining the smallest possible sticks, ensuring that larger sticks (which have a higher combination cost) are combined fewer times.

Algorithm Walkthrough: Using the input [2, 4, 3]:

- Initialize min-heap with the stick lengths: heap = [2, 3, 4]
- Take out the smallest two sticks: 2, 3. Their combined length is 5, adding 5 to the total cost. Push 5 back into the heap.
- Heap now looks like: [4, 5]
- Again, take out the smallest two sticks: 4, 5. Their combined length is 9, adding 9 to the total cost.
- Total cost = 5 + 9 = 14

Code

Here is the code for this algorithm:



```java
import java.util.PriorityQueue;

public class Solution {
    public int connectSticks(int[] sticks) {
        // Initialize a priority queue
        PriorityQueue<Integer> minHeap = new PriorityQueue<>();

        // Add all sticks to the priority queue
        for (int stick : sticks) {
            minHeap.add(stick);
        }

        int cost = 0;

        // Continue until there's only one stick left
        while (minHeap.size() > 1) {
            int first = minHeap.poll();  // Get the smallest stick
            int second = minHeap.poll(); // Get the second smallest stick

            // Combine the two sticks
            int combined = first + second;
```

You must run your code first

Time Complexity:

1. **Heap Construction**: Constructing a heap from an array has a worst-case time complexity of (O(n log n)), where (n) is the number of elements (sticks) in the array.
2. **Heap Operations**:
   - **pop operation**: Takes (O(log n)) time because in the worst case we might need to rearrange the heap after removing the smallest element.
   - **push operation**: Also takes (O(log n)) time, as we might need to rearrange the heap after adding an element.

Since we pop two elements and push one element back in each iteration of the loop, and this is done until there's only one element left in the heap, we do these operations roughly (n-1) times.

So, for (n-1) iterations with 2 pops and 1 push per iteration, the time complexity becomes (O(3(n-1) log n)), which simplifies to (O(n log n)).

3. **Overall Time Complexity**: Combining heap construction and operations, the time complexity remains (O(n log n)).

Space Complexity:

1. **Heap Storage**: We store all (n) sticks in the heap, so the space complexity is (O(n)).
2. **Auxiliary Space**: We use constant extra space for variables, so the auxiliary space complexity is (O(1)).
3. **Overall Space Complexity**: Combining heap storage and auxiliary space, the space complexity is (O(n) + O(1)), which simplifies to (O(n)).

Summary:

- **Time Complexity:** (O(n log n))
- **Space Complexity:** (O(n))

← Previous
Minimum Cost to Connect Sticks(Medium)

Next →
Find The Median Of A Number Stream (Medium)

☐ Mark as Completed